

APEXPLANET

CYBERSECURITY & ETHICAL HACKING INTERNSHIP PROGRAM

TASK – 3

Web Application Security

Practical Task Report

Name	K R Sri Krishna Tulasi
ID	APSPL2643218
Course / Program	Cybersecurity & Ethical Hacking Internship Program
Date	17 September 2026
Task	Task – 3

1. Executive Summary

This report documents the security testing performed against a locally hosted instance of Damn Vulnerable Web Application (DVWA) as part of Task 3 of the ApexPlanet Cybersecurity Internship. The assessment focused on SQL Injection (SQLi), Cross-Site Scripting (XSS), and Cross-Site Request Forgery (CSRF), with supporting testing evidence collected during the practical exercises.

The testing demonstrated that the intentionally vulnerable DVWA environment accepted malicious or unauthorized input under the Low security configuration. SQL Injection was used to manipulate database queries and retrieve user records including stored password hashes. Stored and Reflected XSS payloads were executed in the application. A tokenless CSRF request was accepted at the vulnerable security level. Mitigation controls were then demonstrated using parameterized/prepared SQL statements, Content Security Policy (CSP), and CSRF token validation respectively.

All testing described in this report was conducted against the local DVWA laboratory environment. The objective was educational security validation rather than testing of external systems.

2. Testing Environment

Component	Configuration / Purpose
Operating System	Kali Linux
Application	Damn Vulnerable Web Application (DVWA)
Web Server	Apache
Database	MariaDB / MySQL-compatible database
PHP	PHP 8.4.24
Testing Tool	Burp Suite Community Edition (supporting Task 3 testing)
Target	Localhost DVWA laboratory environment

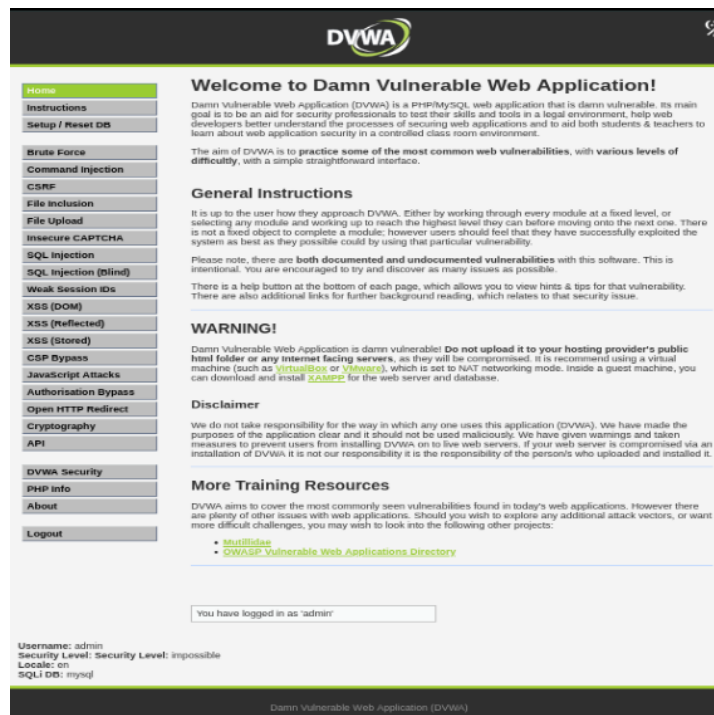


Figure 1. DVWA welcome page and local training environment.

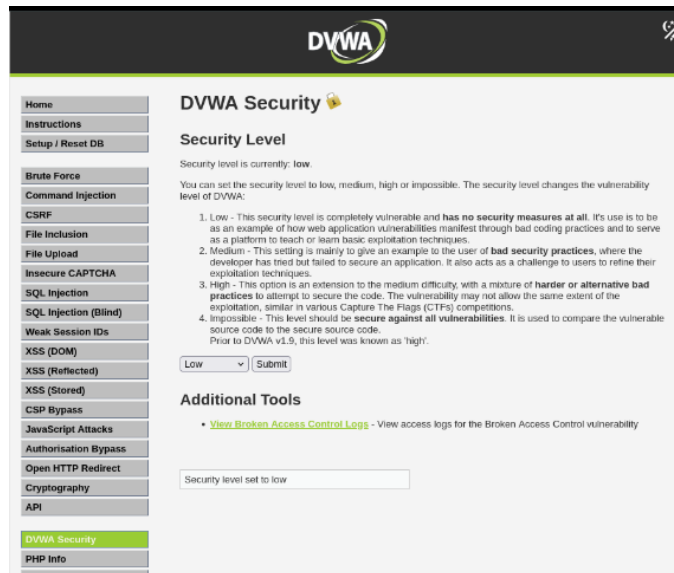


Figure 2. DVWA Security page showing the selected security configuration.

3. Methodology

Testing was performed by selecting the relevant DVWA vulnerability module, establishing a normal baseline request, supplying controlled test input, observing the application response, and then validating the corresponding mitigation where required by the task.

- Establish a baseline using legitimate input.
- Set the relevant DVWA security level to Low for vulnerability demonstration.
- Submit controlled proof-of-concept input in the local lab.
- Record application behavior and supporting source-code/configuration evidence.
- Apply the prescribed mitigation and repeat the test where applicable.
- Capture screenshots at significant testing stages for inclusion in the report.

4. SQL Injection (SQLi)

4.1 Objective

To identify and demonstrate SQL Injection in DVWA, extract user information including stored password hashes, and demonstrate protection using prepared/parameterized statements.

4.2 Baseline Testing

The SQL Injection module was accessed in the DVWA environment. A normal numeric user identifier was submitted first to establish expected application behavior.

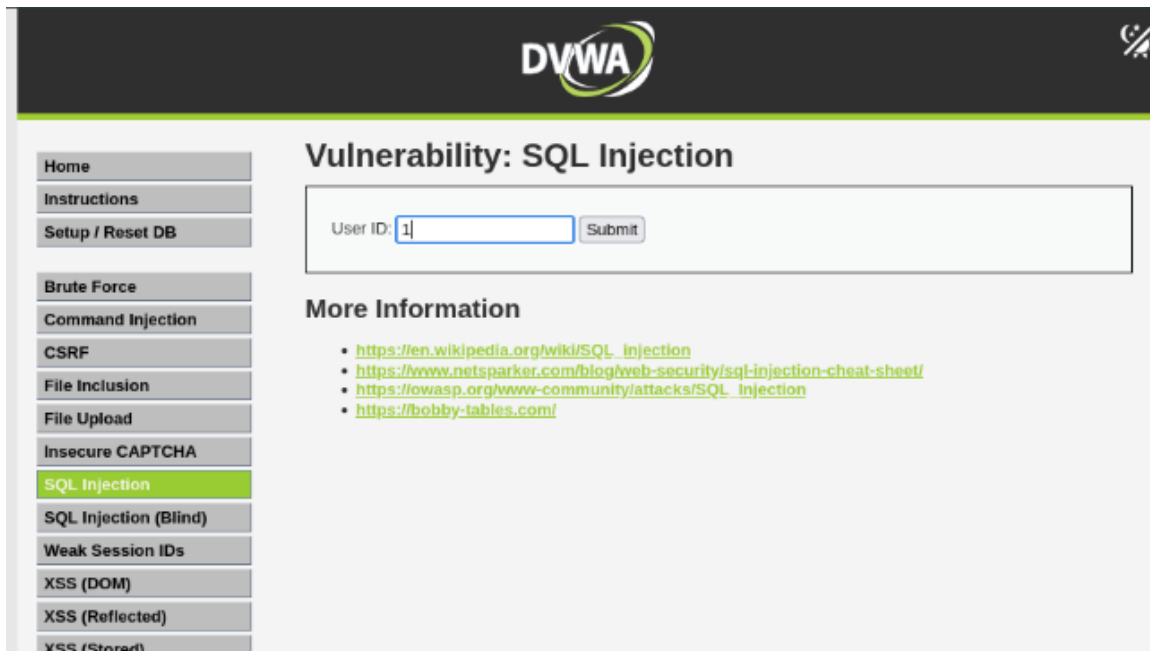


Figure 3. SQL Injection module baseline with a normal user identifier.

4.3 SQL Injection Exploitation

A controlled SQL Injection input was then supplied. The application returned multiple records rather than only the single record associated with the intended identifier, demonstrating that the input was being interpreted as part of the SQL query.



Figure 4. SQL Injection input and the resulting abnormal database response.

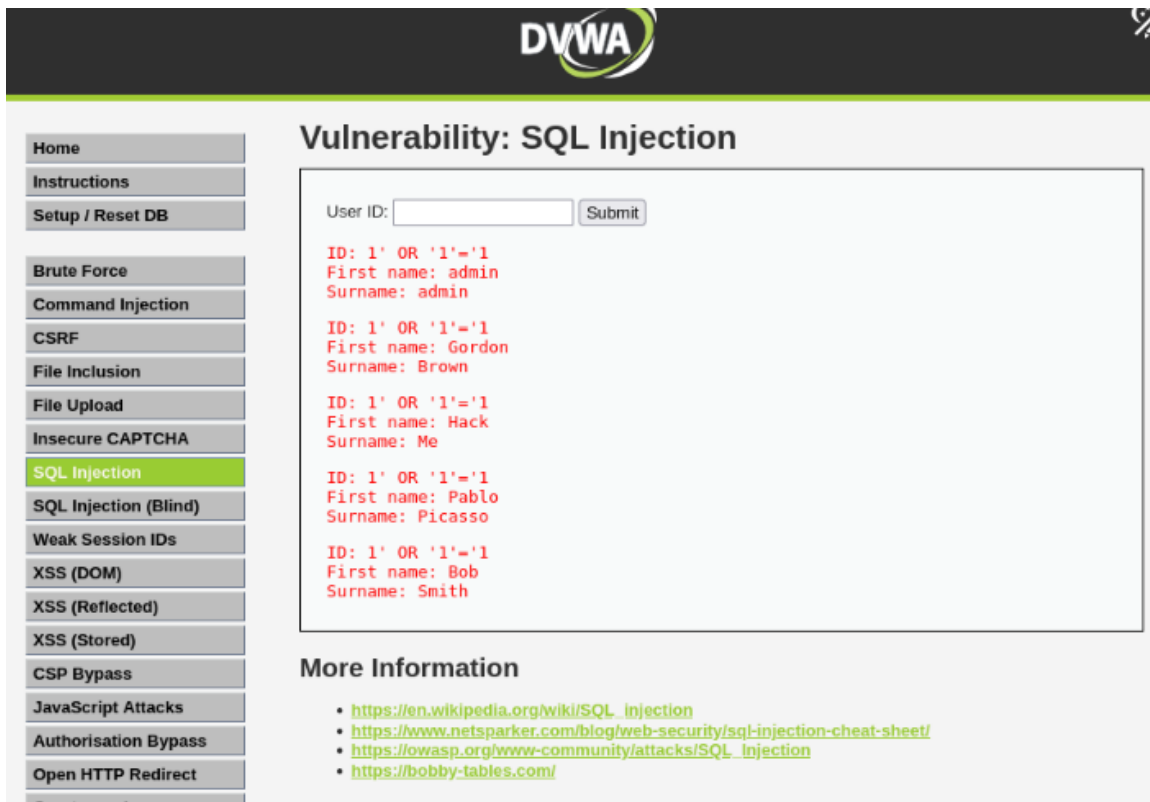


Figure 5. Successful SQL Injection response showing multiple user records.

A UNION-based test was subsequently used to retrieve the username and password fields from the application database. The evidence shows stored password values in hash form rather than plaintext credentials.

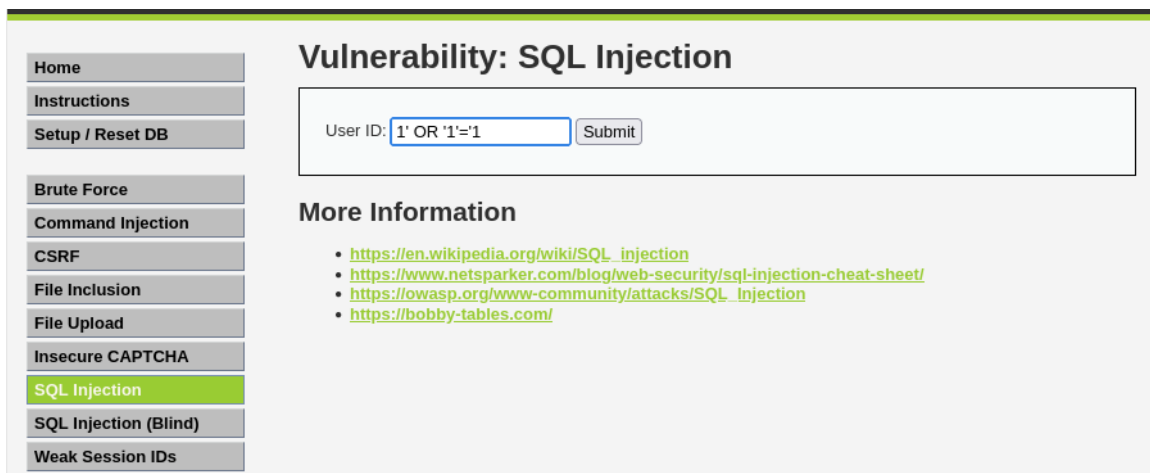


Figure 6. UNION-based SQL Injection input used to retrieve database fields.



Figure 7. SQL Injection response displaying extracted username/password-hash data.

4.4 Vulnerable Implementation

The captured DVWA source demonstrates direct incorporation of the user-controlled identifier into the SQL query. This construction allows crafted input to alter the intended query logic.

```

<?php
if (isset($_GET['Submit'])) {
    // Check Anti-CSRF token
    checkToken($_REQUEST['user_token'], $_SESSION['session_token'], 'index.php');

    // Get input
    $id = $_GET['id'];

    // Was a number entered?
    if (is_numeric($id)) {
        $id = intval($id);
        switch ($DVWA['SQL_DB']) {
            case MYSQL:
                // Check the database
                $data = $db->prepare("SELECT first_name, last_name FROM users WHERE user_id = (:id) LIMIT 1");
                $data->bindParam(':id', $id, PDO::PARAM_INT);
                $data->execute();
                $row = $data->fetch();

                // Make sure only 1 result is returned
                if ($data->rowCount() == 1) {
                    // Get values
                    $first = $row['first_name'];
                    $last = $row['last_name'];

                    // Feedback for end user
                    $html = "<pre>ID: ($id)<br />First name: ($first)<br />Surname: ($last)</pre>";
                }
                break;
            case SQLITE:
                global $sqlite_db_connection;
                $stmt = $sqlite_db_connection->prepare("SELECT first_name, last_name FROM users WHERE user_id = :id LIMIT 1");
                $stmt->bindValue(':id', $id, SQLITE3_INTEGER);
                $result = $stmt->execute();
                $result->finalize();
                if ($result == false) {
                    // There is no way to get the number of rows returned
                    // This checks the number of columns (not rows) just
                    // as a precaution, but it won't stop someone dumping
                    // multiple rows and viewing them one at a time.
                    $num_columns = $result->numColumns();
                    if ($num_columns == 2) {
                        $row = $result->fetchArray();

                        // Get values
                        $first = $row['first_name'];
                        $last = $row['last_name'];

                        // Feedback for end user
                        $html = "<pre>ID: ($id)<br />First name: ($first)<br />Surname: ($last)</pre>";
                    }
                }
                break;
        }
    }
}

// Generate Anti-CSRF token
generateSessionToken();

```

Figure 8. DVWA source-code evidence for the vulnerable SQL Injection implementation.

4.5 Mitigation — Prepared Statements

Prepared/parameterized statements separate SQL code from user-supplied values. The user input is therefore bound as data rather than being concatenated directly into the SQL statement. DVWA's higher-security implementation also uses input handling and prepared-statement mechanisms.

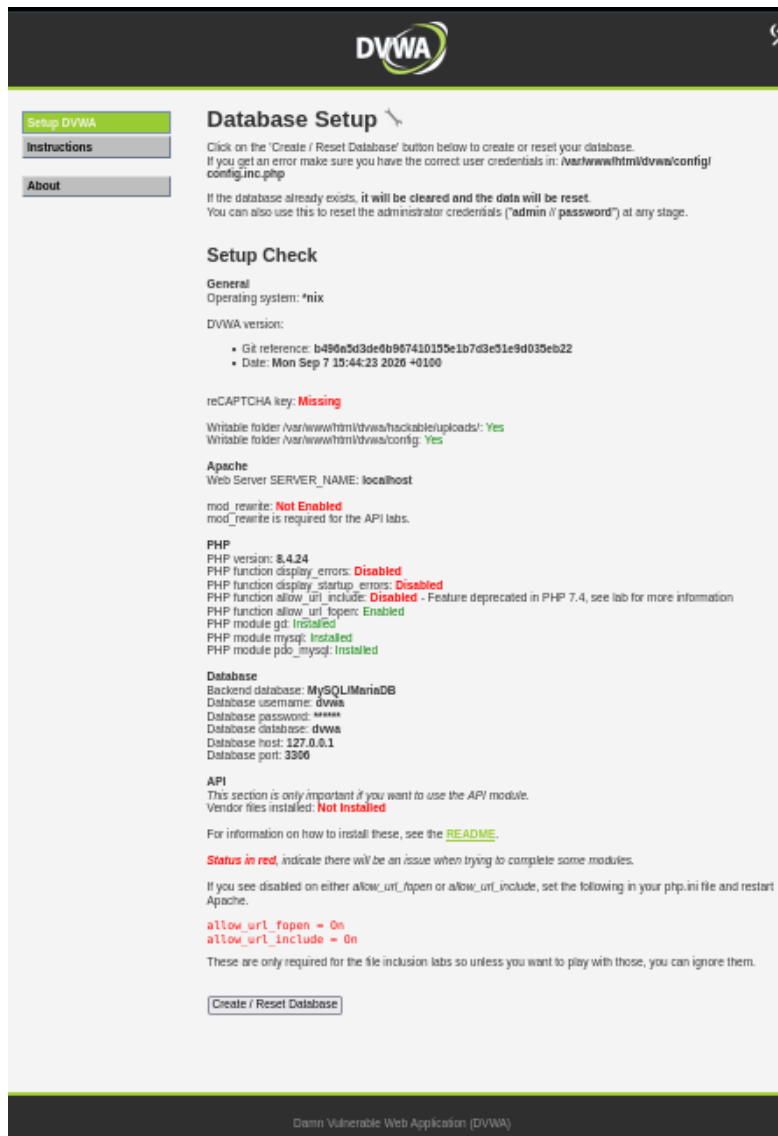


Figure 9. DVWA database/security configuration evidence used during the testing process.

4.6 Impact

- Unauthorized manipulation of database queries.
- Disclosure of application user records.
- Exposure of stored password hashes.
- Potential compromise of confidentiality and integrity if similar weaknesses exist in a production application.

4.7 Result

SQL Injection was successfully demonstrated in the local DVWA environment, followed by verification of the prepared-statement-based protection.

5. Cross-Site Scripting (XSS)

5.1 Objective

To demonstrate both Stored XSS and Reflected XSS in DVWA and validate Content Security Policy (CSP) as a mitigation.

5.2 Stored XSS

The Stored XSS module was first observed under the vulnerable configuration. A harmless JavaScript proof-of-concept payload was entered into the guestbook message field. The resulting JavaScript alert demonstrated execution in the application context.

Figure 10. Stored XSS baseline — guestbook input form.

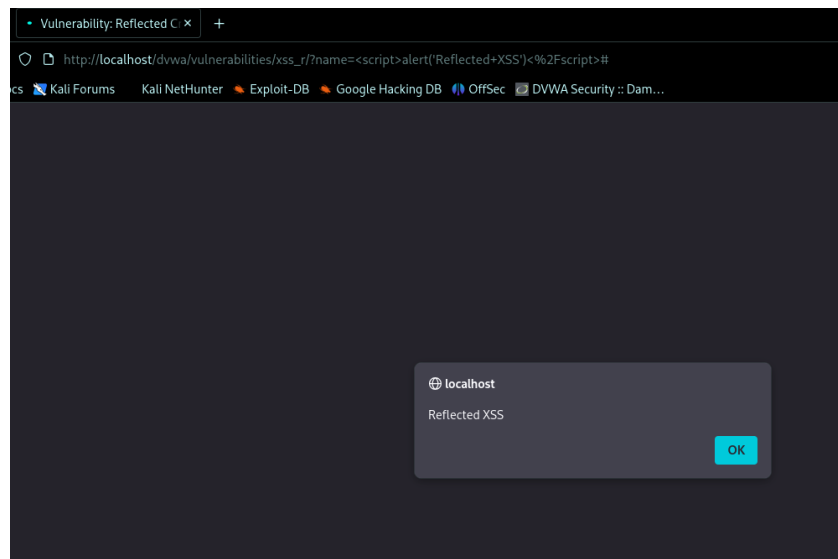


Figure 11. Stored XSS payload execution demonstrated by the JavaScript alert.

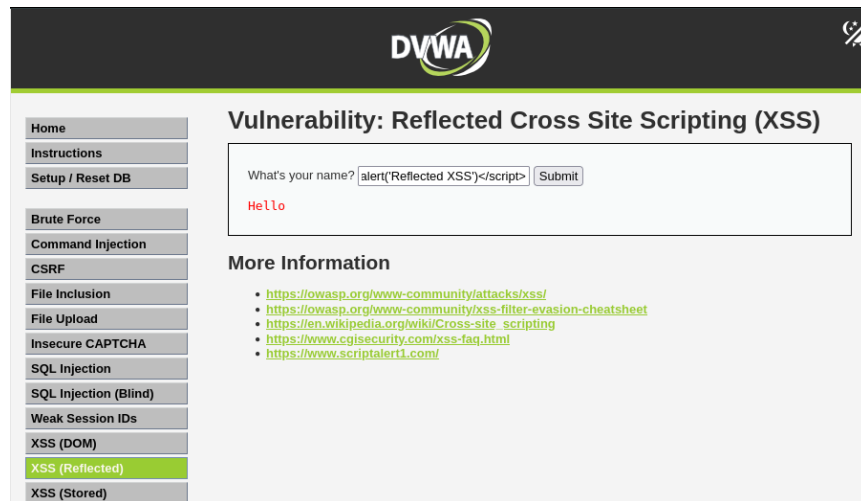


Figure 12. Stored XSS entry remaining in the guestbook after submission.

5.3 Reflected XSS

The Reflected XSS module was tested with a controlled JavaScript proof-of-concept payload. The payload was reflected by the application and executed in the browser, confirming the reflected nature of the vulnerability.

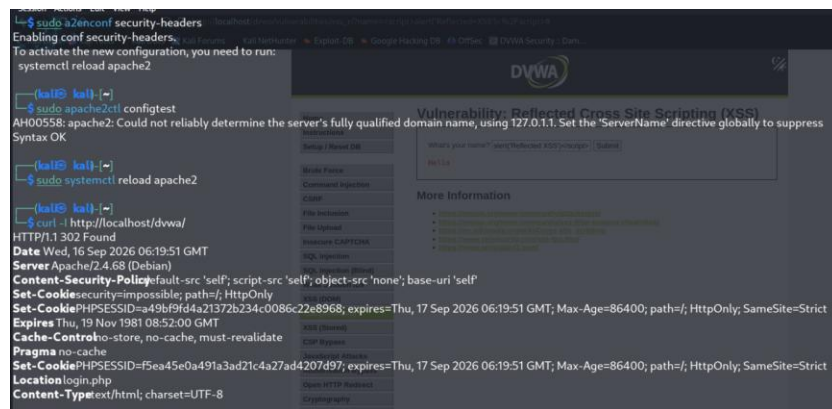


Figure 13. Reflected XSS baseline input form.

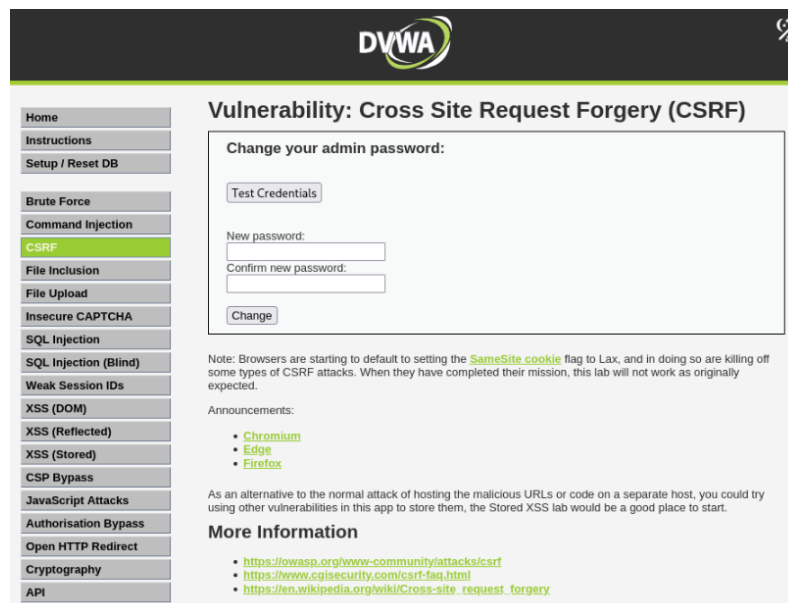


Figure 14. Reflected XSS payload execution demonstrated by the browser alert.

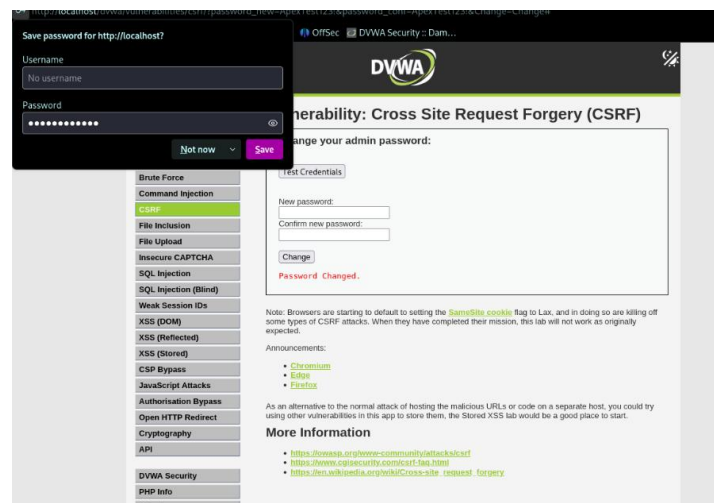


Figure 15. Reflected XSS response showing the supplied input reflected by the application.

5.4 Mitigation — Content Security Policy

A Content Security Policy was configured at the Apache web-server level. The policy restricts script sources and disallows object content, providing an additional layer of protection against script injection.

```
Content-Security-Policy: default-src 'self'; script-src 'self'; object-src 'none'; base-uri 'self'
```

The same reflected JavaScript test was then checked after applying the policy. The security-header configuration was also verified through an HTTP response inspection.

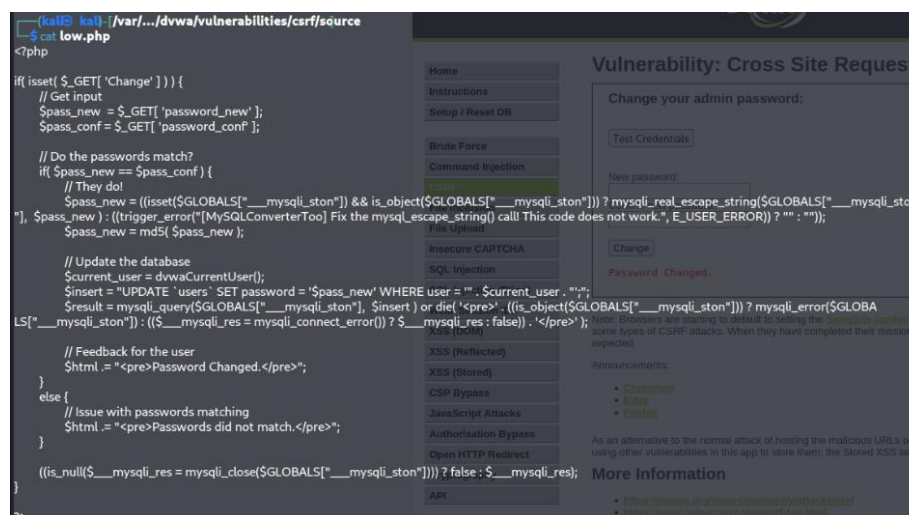


Figure 16. Apache security-header configuration and local HTTP response showing the CSP header.

5.5 Impact

- Execution of attacker-controlled JavaScript in a victim's browser context.
- Potential manipulation of displayed page content.
- Potential exposure of information available to the affected browser context.
- Loss of trust in application content when untrusted input is rendered without adequate protection.

5.6 Result

Both Stored and Reflected XSS were successfully demonstrated in the DVWA laboratory. CSP was configured and the security-header response was verified as part of the mitigation demonstration.

6. Cross-Site Request Forgery (CSRF)

6.1 Objective

To demonstrate a CSRF condition in DVWA in which a password-changing request can be submitted without a CSRF token, and to verify rejection when token validation is enforced.

6.2 Baseline and Legitimate Password Change

The CSRF module was accessed and its password-change functionality was observed. A legitimate password change was performed to establish the expected behavior of the endpoint.

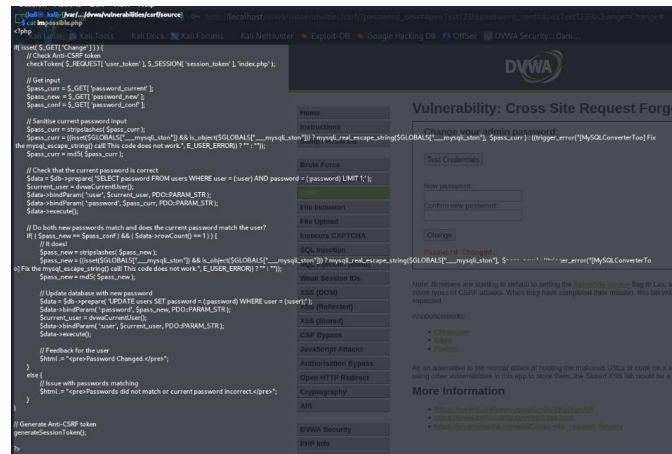


Figure 17. CSRF module baseline showing the password-change form.

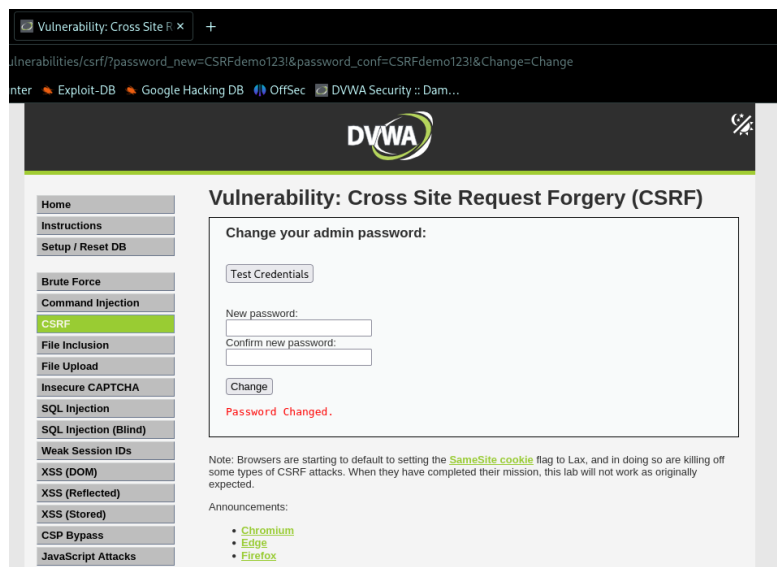


Figure 18. Legitimate password change successfully processed by DVWA.

6.3 Vulnerable Implementation

The vulnerable source code accepts the password-change parameters and performs the update without first requiring a CSRF token tied to the user's session.

Vulnerability: Cross Site Request Forgery (CSRF)

Change your admin password:

Current password:

New password:

Confirm new password:

Note: Browsers are starting to default to setting the [SameSite cookie](#) flag to Lax, and in doing so are killing off some types of CSRF attacks. When they have completed their mission, this lab will not work as originally expected.

Announcements:

- [Chromium](#)
- [Edge](#)
- [Firefox](#)

As an alternative to the normal attack of hosting the malicious URLs or code on a separate host, you could try using other vulnerabilities in this app to store them, the Stored XSS lab would be a good place to start.

More Information

- <https://owasp.org/www-community/attacks/csrf>
- <https://www.cgisecurity.com/csrf-faq.html>
- https://en.wikipedia.org/wiki/Cross-site_request_forgery

CSRF token is incorrect

Username: admin
Security Level: Security Level: impossible
Locale: en
SQL DB: mysql

Figure 19. DVWA vulnerable CSRF source-code implementation.

6.4 Token-Based Protection

The protected implementation checks a submitted user token against the expected session token before processing the password change. This prevents a request lacking the required token from being accepted.

Vulnerability: Stored Cross Site Scripting (XSS)

Name *

Message *

localhost
Stored XSS

- <https://www.cgisecurity.com/xss-faq.html>
- <https://www.scriptalert1.com/>

Username: admin
Security Level: Security Level: impossible
Locale: en
SQL DB: mysql

Figure 20. DVWA protected CSRF implementation showing token validation.

6.5 CSRF Demonstration

A controlled tokenless password-change request was submitted in the vulnerable configuration and was accepted, demonstrating the absence of token enforcement at that security level.

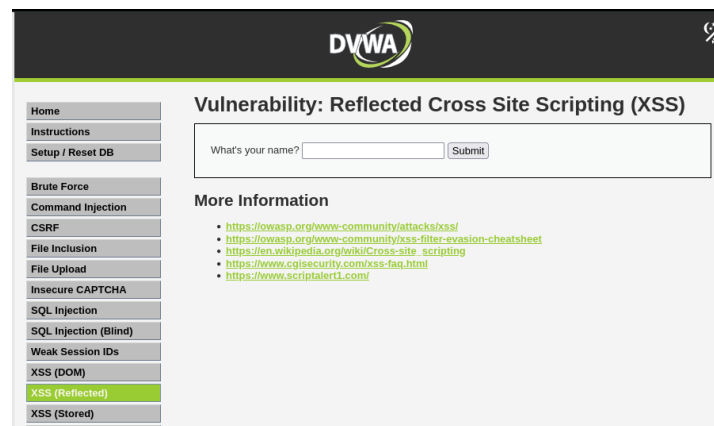


Figure 21. Tokenless password-change request accepted under the vulnerable configuration.

The same type of tokenless request was then tested under the protected configuration. The application rejected the request and reported that the CSRF token was incorrect.

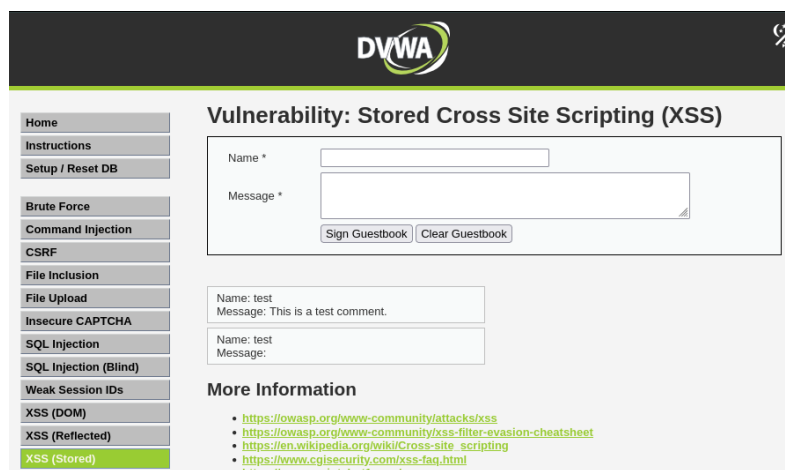


Figure 22. Protected configuration rejecting the tokenless request because the CSRF token is incorrect.

6.6 Impact

- Unauthorized state-changing requests may be performed through a vulnerable endpoint.
- An attacker could potentially cause a victim's authenticated session to submit an unwanted password change.
- Account integrity may be affected when sensitive actions are not protected by request-origin validation or anti-CSRF tokens.

6.7 Result

The vulnerable CSRF behavior was demonstrated and the corresponding token-based protection was verified by the application's rejection of a tokenless request.

7. Findings Summary

Finding	Area	Demonstration	Mitigation / Control
SQL Injection	Database input handling	Query manipulation and user/hash disclosure	Prepared/parameterized statements
Stored XSS	Guestbook input/output	Persistent JavaScript execution	Output encoding + CSP defense in depth
Reflected XSS	Request parameter/output	Reflected JavaScript execution	Output encoding + CSP defense in depth
CSRF	Password-change endpoint	Tokenless state-changing request accepted at Low	Session-bound CSRF token validation

8. Security Recommendations

SQL Injection: Use prepared statements/parameterized queries throughout the application. Validate input according to expected type and apply least-privilege database permissions.

Stored/Reflected XSS: Perform context-appropriate output encoding, validate input where appropriate, and maintain a restrictive CSP as defense in depth.

CSRF: Require unpredictable, session-bound CSRF tokens for state-changing requests and use appropriate cookie protections such as SameSite where applicable.

Security Headers: Maintain security headers at the web-server/application layer and periodically verify their effective HTTP response values.

Error Handling: Avoid exposing detailed database or application errors to end users; log diagnostic information securely on the server.

Authentication: Use strong password hashing and secure session-management practices in production systems.

9. Conclusion

The Task 3 web-application security exercises were successfully demonstrated in a controlled DVWA laboratory environment. SQL Injection, Stored XSS, Reflected XSS, and CSRF weaknesses were reproduced under the vulnerable configuration, while the prescribed mitigation concepts were demonstrated through prepared statements, CSP, and CSRF token validation. The accompanying evidence provides a traceable record of the testing process and observed results.

10. Evidence Index

The report incorporates the screenshots supplied in the Task 3 evidence document. The figures below provide a quick reference to the principal evidence categories.

Evidence Category	Report Figures
SQL Injection	Figures 3–9
Stored XSS	Figures 10–12
Reflected XSS	Figures 13–16
CSRF	Figures 17–22
Security Header verification	Figure 16